DESIGN AND ANALYSIS OF ALGORITHMS

USING DIVIDE AND CONQUER TECHNIQUES

An Assignment Report

A report submitted in partial fulfilment of the requirements for the course

CSA0607-DESIGN AND ANALYSIS OF ALGORITHMS – ASSIGNMENT 3 (CO3)

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Muthupandi C (192565037)

Natesa Krishnan V (192511311)

Under the Supervision of

Dr. A.Sairam

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

OCTOBER 2026

CONTENTS

- Assignment Background and Objectives
- Q1 – Decomposition and Elimination in Ordered Data
- Q2 – The Quality of Decomposition in Sorting
- Q3 – Karatsuba Multiplication
- Q4 – Strassen's Matrix Multiplication
- Q5 – Closest Pair Using Divide and Conquer
- Q6 – Advanced Divide-and-Conquer Mini-Investigations
- Q7 – Design Divide and Conquer from First Principles
- Complexity Summary
- Conclusion
- Appendix – Python Execution Outputs and Submission Evidence

Assignment Background and Objectives

This pair assignment investigates how divide-and-conquer (D&C) algorithms split a problem into smaller subproblems, solve the subproblems, and combine their results. The work covers ordered-data operations, sorting, integer multiplication, matrix multiplication, geometric search, selection, subset sum, and a new D&C design. Each program is implemented in Python and includes a reproducible demonstration.

Pair-work responsibilities follow the assignment brief. Student A is Muthupandi C (192565037), and Student B is Natesa Krishnan V (192511311). Both students are responsible for understanding all seven questions.

Q1 – Decomposition and Elimination in Ordered Data

Part A – Maximum and Minimum. The input is divided into two halves until a one-element or two-element base case is reached. Each recursive call returns a minimum and maximum. The combine step compares the two local minima and the two local maxima. For n elements, the comparison count is approximately $3n/2 - 2$ when n is a power of two; independently finding minimum and maximum uses $2n - 2$ comparisons.

Part B – Binary Search. The sorted array is inspected at its middle index. If the middle value is smaller than the target, the left half is eliminated; otherwise, the right half is eliminated. The invariant is that if the target exists, it remains within the active interval. The loop terminates when the target is found or the interval becomes empty.

Time complexity: Max/Min O(n); Binary Search O(log n). Auxiliary recursion space for Max/Min is O(log n); Binary Search uses O(1) iterative space.

Execution Output

```
Q1 MAX-MIN
Data: [18, 4, 27, 9, 2, 31, 12, 6]
Minimum=2, Maximum=31, Comparisons=10

Q1 BINARY SEARCH
Sorted data: [2, 4, 6, 9, 12, 18, 27, 31]
Target=12, Index=4, Comparisons=5
Target=20, Index=-1, Comparisons=6
```

Q2 – The Quality of Decomposition in Sorting

Merge Sort divides the list into two nearly equal halves, recursively sorts each half, and merges them. Its decomposition is balanced and predictable, giving O(n log n) time and O(n) auxiliary space.

Quick Sort uses the last element as its pivot in this implementation. Values less than or equal to the pivot form the left partition; larger values form the right partition. Balanced partitions give O(n log n) average time, while repeated highly unbalanced partitions can produce $O(n^2)$ worst-case time and

O(n) recursion depth. The supplied script records maximum recursion depth for the demonstration inputs.

For unknown input characteristics, pivot behaviour, input order, duplicate frequency, memory limits, stability requirements, and worst-case guarantees should be considered before choosing a sorting strategy.

Execution Output

```
mixed:
Merge Sort: [2, 5, 12, 19, 27, 31, 38, 44]
Quick Sort: [2, 5, 12, 19, 27, 31, 38, 44] | maximum recursion depth: 5
Outputs match: True
ascending:
Merge Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Quick Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] | maximum recursion depth: 10
Outputs match: True
```

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication

Write two n-digit integers as x = a·10^m + b and y = c·10^m + d. Conventional multiplication computes ac, ad, bc, and bd. Karatsuba computes z2 = ac, z0 = bd, and z1 = (a+b)(c+d) − z2 − z0, reducing four recursive multiplications to three.

The recurrence is T(n) = 3T(n/2) + O(n), so the time complexity is $O(n^{(\log_2 3)})$, approximately O(n^1.585). The implementation compares the exact result with Python's standard integer multiplication. Karatsuba may be slower for small values because recursive calls, splitting, additions, and conversions introduce overhead.

Execution Output

```
Q3 KARATSUBA MULTIPLICATION
Standard result: 12193263113702107135954925 3925
Karatsuba result: 12193263113702107135954925 3925
Correct: True
Recurrence: T(n) = 3T(n/2) + O(n)
Complexity: O(n^(log2 3)) approximately O(n^1.585)
```

Q4 – The Reality of Strassen's Matrix Multiplication

Conventional block multiplication uses eight block multiplications. Strassen's transformation forms seven products and combines them using matrix additions and subtractions. Its recurrence is T(n) = 7T(n/2) + $O(n^2)$, giving $O(n^{(\log_2 7)})$, approximately O(n^2.807).

The implementation pads non-power-of-two square matrices with zeros to the next power-of-two dimension, performs Strassen multiplication, and removes the padded rows and columns. Padding and extra additions create overhead, so conventional multiplication can remain preferable for small

matrices or memory-constrained systems. The program checks Strassen's output against conventional multiplication.

Execution Output

```
Q4 STRASSEN MATRIX MULTIPLICATION
Conventional: [[30, 24, 18], [84, 69, 54], [138, 114, 90]]
Strassen: [[30, 24, 18], [84, 69, 54], [138, 114, 90]]
Correct: True
Recurrence: T(n) = 7T(n/2) + O(n^2)
Complexity: O(n^(log2 7)) approximately O(n^2.807)
```

Q5 – Upgrading Closest Pair Using Divide and Conquer

The points are sorted by x-coordinate and divided into left and right groups. Each recursive call returns the smallest distance and corresponding pair in its region. The combine stage considers points in a vertical strip around the dividing line. The strip is ordered by y-coordinate; only nearby following points need to be checked because a better cross-boundary pair must lie within the current best distance in both coordinates.

The brute-force baseline checks every pair and takes $O(n^2)$ time. The standard geometric D&C method achieves O(n log n) when x- and y-sorted lists are maintained efficiently. This teaching implementation sorts the strip during recursive calls, so its practical worst-case bound can include additional sorting overhead; the output is validated against brute force.

Execution Output

```
Q5 CLOSEST PAIR
Brute force: (1.4142135623730951, ((2, 3), (3, 4)), 21)
Divide and conquer: (1.4142135623730951, ((2, 3), (3, 4)), 6)
Same minimum distance: True
```

Q6 – Advanced Divide-and-Conquer Mini-Investigations

Student A – Median of Medians. The input is divided into groups of at most five. The median of each group is found, and the median of those medians is selected as a pivot. Partitioning around this pivot guarantees a sufficiently large fraction of elements can be discarded, providing worst-case O(n) selection time.

Student B – Meet-in-the-Middle. The subset-sum input is divided into two halves. All subset sums are generated for each half. One list is sorted, and a two-pointer search checks whether a left sum and right sum total the target. This reduces the exponential search from roughly O(2^n) enumeration to about O(2^(n/2)) generated sums, at the cost of O(2^(n/2)) memory. The implementation uses sorting for the right-half sums.

Execution Output

```
Q6 MEDIAN OF MEDIANS
5-th smallest (1-based): 9
```

Q6 MEET-IN-THE-MIDDLE
Values: [3, 34, 4, 12, 5, 2] Target: 9
Subset exists: False
Time: O(2^(n/2) log(2^(n/2))) for this sorting-based combination
Space: O(2^(n/2))

Q7 – Design Divide and Conquer from First Principles

Chosen problem: Maximum Contiguous Subarray Sum. Student A and Student B should retain their own initial proposals in the final submission and then jointly agree on the final design.

Decomposition: split the array into left and right halves. Base case: a one-element array returns that element. Each recursive call returns the maximum subarray sum entirely within its half. Combine: calculate the best suffix sum ending at the midpoint and the best prefix sum beginning immediately after it; their sum is the best crossing subarray. Return the maximum of left, right, and crossing results.

Recurrence: T(n) = 2T(n/2) + O(n), which solves to O(n log n). Recursion space is O(log n). The implementation compares its answer with a brute-force baseline on a sample containing positive and negative values.

Execution Output

Q7 DESIGN FROM FIRST PRINCIPLES: MAXIMUM SUBARRAY
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Divide-and-conquer result: 6
Brute-force result: 6
Correct: True
Recurrence: T(n) = 2T(n/2) + O(n)
Time: O(n log n); auxiliary recursion space: O(log n)

Complexity Summary

Algorithm	Time Complexity	Auxiliary Space
Q1 Max/Min	O(n)	O(log n) recursion
Q1 Binary Search	O(log n)	O(1)
Q2 Merge Sort	O(n log n)	O(n)
Q2 Quick Sort	Average O(n log n); worst $O(n^2)$	Average O(log n); worst O(n)
Q3 Karatsuba	O(n^1.585)	Recursion-dependent
Q4 Strassen	O(n^2.807)	Recursion and matrix temporaries
Q5 Closest Pair	O(n log n) with efficient strip handling	O(n) typical
Q6 Median of Medians	O(n) worst case	O(n) in this list-based implementation
Q6 Meet-in-the-Middle	O(2^(n/2) log 2^(n/2))	O(2^(n/2))
Q7 Maximum Subarray D&C	O(n log n)	O(log n) recursion

Pair Contribution Record

The following is a proposed primary-responsibility allocation based on the assignment brief. Both students must participate in and understand every question. Confirm that this reflects the work actually completed before submission.

Question	Student A – Muthupandi C	Student B – Natesa Krishnan V	Joint Work
Q1	Max/Min design and analysis	Binary Search elimination logic	Implementation and testing
Q2	Quick Sort experiments	Merge Sort comparison analysis	Implement both sorts
Q3	Karatsuba derivation	Crossover experiment	Standard and Karatsuba implementation
Q4	Experimental/engineering analysis	Algebraic design and correctness	Implement and verify
Q5	D&C decomposition design	Comparative evidence	Implement and compare
Q6	Median of Medians	Meet-in-the-Middle	Cross-verification
Q7	Mathematical validation	Experimental validation	Independent proposals and final design

Conclusion

The assignment demonstrates that divide and conquer can reduce computational work by splitting a large problem into smaller subproblems and combining their results. The benefit depends on the quality of the split, the cost of combining results, recursion overhead, and memory requirements. Experimental results should be collected on the submission computer because execution times vary by hardware and software environment.

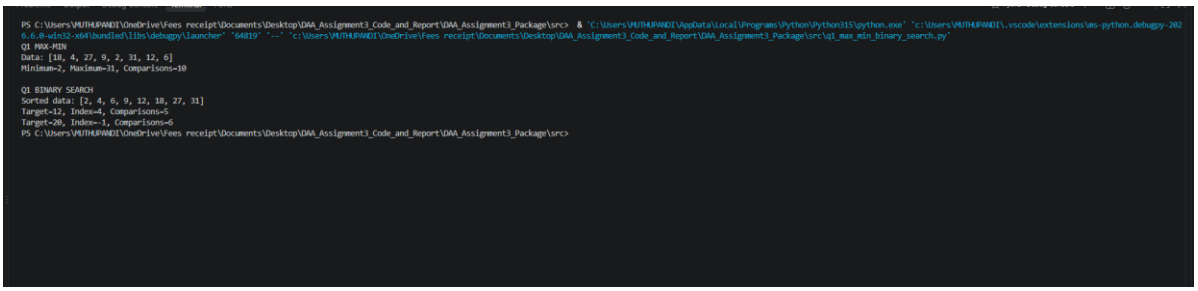
Appendix – Submission Evidence

The ZIP package contains seven independent Python programs and a README. Run each program in VS Code or a terminal and insert genuine screenshots of the output here before final submission. Also retain each student's independent Q7 proposal and update the contribution record to match the actual work.

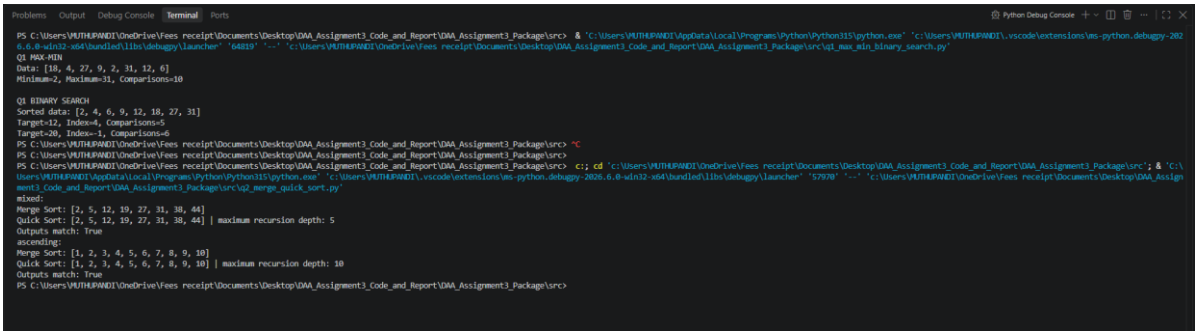
APPENDIX – ACTUAL PYTHON EXECUTION SCREENSHOTS

The following screenshots were supplied from the student's VS Code terminal. They show execution evidence for the DAA Assignment 3 programs. The images are included as provided and are grouped by the questions visible in each terminal run.

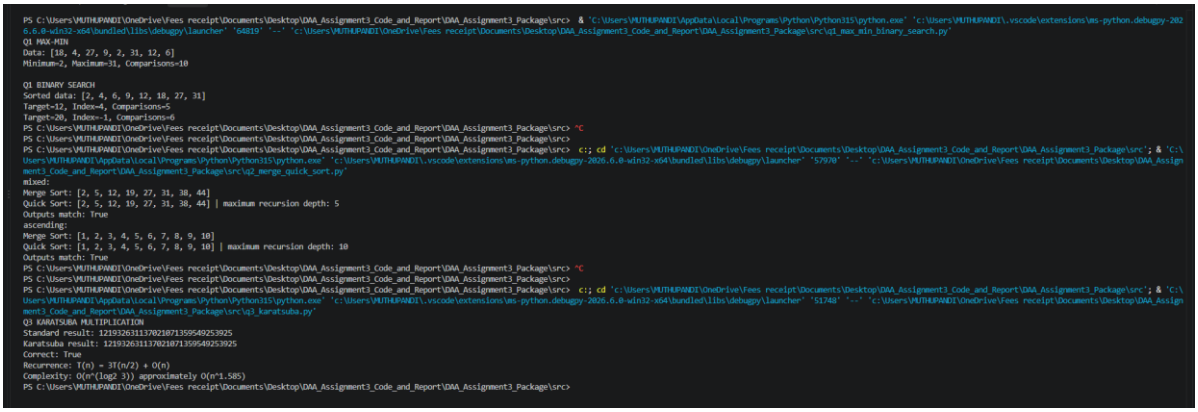
Q1 – Maximum-Minimum and Binary Search



Q2 – Merge Sort and Quick Sort



Q3 and Q4 – Karatsuba and Strassen Multiplication



Q5 – Closest Pair using Divide and Conquer



Q1–Q5 – Combined Terminal Execution Evidence



Q5 and Q6 – Closest Pair and Advanced D&C Investigations

```
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64819' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q1_max_min_binary_search.py'
Q1 MAX-MIN
Data: [18, 4, 27, 9, 2, 31, 12, 6]
Minimum=2, Maximum=31, Comparisons=10

Q1 BINARY SEARCH
Sorted data: [2, 4, 6, 9, 12, 18, 27, 31]
Target=12, Index=4, Comparisons=5
Target=20, Index=-1, Comparisons=6
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57970' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q2_merge_quick_sort.py'
mixed:
Merge Sort: [2, 5, 12, 19, 27, 31, 38, 44]
Quick Sort: [2, 5, 12, 19, 27, 31, 38, 44] | maximum recursion depth: 5
Outputs match: True
ascending:
Merge Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Quick Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] | maximum recursion depth: 10
Outputs match: True
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51748' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q3_karatsuba.py'
Q3 KARATSUBA MULTIPLICATION
Standard result: 121932631137021071359549253925
Karatsuba result: 121932631137021071359549253925
Correct: True
Recurrence: T(n) = 3T(n/2) + O(n)
Complexity: O(n^(log2 3)) approximately O(n^1.585)
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51787' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q4_strassen.py'
Q4 STRASSEN MATRIX MULTIPLICATION
Conventional: [[30, 24, 18], [84, 69, 54], [138, 114, 90]]
Strassen: [[30, 24, 18], [84, 69, 54], [138, 114, 90]]
Correct: True
Recurrence: T(n) = 7T(n/2) + O(n^2)
Complexity: O(n^(log2 7)) approximately O(n^2.807)
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56333' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q5_closest_pair.py'
Q5 CLOSEST PAIR
Brute force: (1.4142135623730951, ((2, 3), (3, 4)), 21)
Divide and conquer: (1.4142135623730951, ((2, 3), (3, 4)), 6)
```

Q7 – Maximum Subarray Design

```
mixed:
Merge Sort: [2, 5, 12, 19, 27, 31, 38, 44]
Quick Sort: [2, 5, 12, 19, 27, 31, 38, 44] | maximum recursion depth: 5
Outputs match: True
ascending:
Merge Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Quick Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] | maximum recursion depth: 10
Outputs match: True
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51748' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q3_karatsuba.py'
Q3 KARATSUBA MULTIPLICATION
Standard result: 121932631137021071359549253925
Karatsuba result: 121932631137021071359549253925
Correct: True
Recurrence: T(n) = 3T(n/2) + O(n)
Complexity: O(n^(log2 3)) approximately O(n^1.585)
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51787' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q4_strassen.py'
Q4 STRASSEN MATRIX MULTIPLICATION
Conventional: [[30, 24, 18], [84, 69, 54], [138, 114, 90]]
Strassen: [[30, 24, 18], [84, 69, 54], [138, 114, 90]]
Correct: True
Recurrence: T(n) = 7T(n/2) + O(n^2)
Complexity: O(n^(log2 7)) approximately O(n^2.807)
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56333' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q5_closest_pair.py'
Q5 CLOSEST PAIR
Brute force: (1.4142135623730951, ((2, 3), (3, 4)), 21)
Divide and conquer: (1.4142135623730951, ((2, 3), (3, 4)), 6)
Same minimum distance: True
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src> ^C
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49679' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q6_advanced_dac.py'
Q6 MEDIAN OF MEDIANS
5-th smallest (1-based): 9

Q6 MEET-IN-THE-MIDDLE
Values: [3, 34, 4, 12, 5, 2] Target: 9
Subset exists: False
Time: O(2^(n/2) log(2^(n/2))) for this sorting-based combination
Space: O(2^(n/2))
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
```

Additional VS Code Execution Evidence

```
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>  c:; cd 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src'; & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe'  'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59309' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src\q7_maximum_subarray.py'
Q7 DESIGN FROM FIRST PRINCIPLES: MAXIMUM SUBARRAY
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Divide-and-conquer result: 6
Brute-force result: 6
Correct: True
Recurrence: T(n) = 2T(n/2) + O(n)
Time: O(n log n); auxiliary recursion space: O(log n)
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\DAA_Assignment3_Code_and_Report\DAA_Assignment3_Package\src>
```